

Python Code Test Document

This PDF is intentionally structured with Python classes, functions, comments, conditionals, loops, database-style operations, and API-style code. It can be used to test RecursiveCharacterTextSplitter and Python-aware splitting.

1. User Model

```
class User:
    """Represents a user in the application."""

    def __init__(self, user_id: int, name: str, email: str):
        self.user_id = user_id
        self.name = name
        self.email = email

    def display_name(self) -> str:
        return f"{self.name} <{self.email}>"

    def is_valid(self) -> bool:
        return bool(self.name and self.email and "@" in self.email)

def create_user(user_id: int, name: str, email: str) -> User:
    user = User(user_id, name, email)

    if not user.is_valid():
        raise ValueError("Invalid user information")

    return user
```

2. Authentication Service

```
class AuthenticationService:
    """Handles login and token generation."""

    def __init__(self, user_repository):
        self.user_repository = user_repository

    def authenticate(self, email: str, password: str) -> str:
        user = self.user_repository.find_by_email(email)

        if user is None:
            raise ValueError("User not found")

        if not self.verify_password(password, user.password_hash):
            raise ValueError("Invalid credentials")

        return self.generate_token(user)

    def verify_password(self, password: str, password_hash: str) -> bool:
        return password_hash == hash_password(password)

    def generate_token(self, user: User) -> str:
        payload = {
            "user_id": user.user_id,
            "email": user.email,
        }
        return encode_jwt(payload)

def hash_password(password: str) -> str:
    import hashlib
```

```
return hashlib.sha256(password.encode()).hexdigest()
```

```
def encode_jwt(payload: dict) -> str:  
    return f"jwt-token-for-{payload['user_id']}"
```

3. Product Service

```
class ProductService:  
    """Provides product-related operations."""  
  
    def __init__(self, product_repository):  
        self.product_repository = product_repository  
  
    def get_product(self, product_id: int):  
        product = self.product_repository.find_by_id(product_id)  
  
        if product is None:  
            raise LookupError("Product not found")  
  
        return product  
  
    def search_products(self, query: str, limit: int = 20):  
        products = self.product_repository.search(query)  
  
        return products[:limit]  
  
    def calculate_discount(self, price: float, percentage: float) -> float:  
        if percentage < 0 or percentage > 100:  
            raise ValueError("Discount must be between 0 and 100")  
  
        discount = price * (percentage / 100)  
        return price - discount
```

4. Repository Pattern

```
class UserRepository:  
    """Example repository responsible for user persistence."""  
  
    def __init__(self, database):  
        self.database = database  
  
    def find_by_id(self, user_id: int):  
        query = "SELECT * FROM users WHERE id = ?"  
        return self.database.execute(query, [user_id])  
  
    def find_by_email(self, email: str):  
        query = "SELECT * FROM users WHERE email = ?"  
        return self.database.execute(query, [email])  
  
    def save(self, user: User):  
        query = """  
        INSERT INTO users (id, name, email)  
        VALUES (?, ?, ?)  
        """  
        return self.database.execute(  
            query,  
            [user.user_id, user.name, user.email],  
        )
```

5. API Handler

```
def get_user_handler(request, user_repository):  
    """Example API endpoint for retrieving a user."""  
  
    user_id = request.get("user_id")
```

```

if not user_id:
    return {
        "status": 400,
        "message": "user_id is required",
    }

try:
    user = user_repository.find_by_id(int(user_id))

    if user is None:
        return {
            "status": 404,
            "message": "User not found",
        }

    return {
        "status": 200,
        "data": {
            "id": user.user_id,
            "name": user.name,
            "email": user.email,
        },
    }

except ValueError:
    return {
        "status": 400,
        "message": "Invalid user_id",
    }

except Exception as error:
    print(f"Unexpected error: {error}")
    return {
        "status": 500,
        "message": "Internal server error",
    }

```

6. Data Processing

```

def calculate_statistics(numbers: list[float]) -> dict:
    """Calculate basic statistics for a list of numbers."""

    if not numbers:
        return {
            "count": 0,
            "minimum": None,
            "maximum": None,
            "average": None,
        }

    total = 0

    for number in numbers:
        total += number

    return {
        "count": len(numbers),
        "minimum": min(numbers),
        "maximum": max(numbers),
        "average": total / len(numbers),
    }

def process_orders(orders: list[dict]) -> list[dict]:
    """Process orders and calculate totals."""

```

```
processed_orders = []

for order in orders:
    quantity = order.get("quantity", 0)
    price = order.get("price", 0)

    total = quantity * price

    processed_orders.append({
        "order_id": order.get("order_id"),
        "total": total,
        "status": "processed",
    })

return processed_orders
```

7. Application Entry Point

```
def main():
    """Application entry point."""

    users = [
        create_user(1, "Rahul", "rahul@example.com"),
        create_user(2, "Amit", "amit@example.com"),
        create_user(3, "Priya", "priya@example.com"),
    ]

    for user in users:
        print(user.display_name())

    statistics = calculate_statistics([10, 20, 30, 40, 50])

    print("Statistics:")
    print(statistics)

if __name__ == "__main__":
    main()
```